

Call Stack e Event Loop

Objetivo da Aula

Compreender as estruturas de execução síncrona e assíncrona do NodeJS, com base nos componentes de call stack e event loop.

Apresentação

Hoje em dia, é impossível pensar em sistemas para web que não utilizem processos assíncronos em determinados pontos. A plataforma NodeJS oferece um modelo de tratamento de processos que traz grande eficiência, dividindo o processamento síncrono e assíncrono em estruturas bem delineadas.

Precisamos compreender o funcionamento síncrono, com base na pilha de chamadas, ou call stack, e a organização das filas de tratamento para respostas assíncronas, ou callback queue. Não menos importante, devemos compreender a relação do event loop com essas estruturas.

Após estudar o modelo de processamento, será necessário observar como a programação assíncrona é implementada em JavaScript, identificando quais as palavras reservadas e compreendendo o uso de promises e callbacks.

1. Call Stack e Chamadas Síncronas

Uma pilha de chamadas, ou **call stack**, é uma estrutura do tipo **LIFO** que tem como objetivo organizar a ordem de chamadas das funções dentro do fluxo de execução do programa. Não é algo novo, sendo utilizado pelas linguagens de programação para garantir que as instruções sejam executadas na ordem correta e que uma função possa aguardar o término de outra, que tenha sido invocada em seu próprio fluxo de execução.

Cada vez que uma função é invocada, ela é colocada no topo da pilha, sendo retirada ao final de sua execução, com o retorno para o fluxo do chamador, que estará na posição seguinte da pilha. No caso de funções recursivas, a cada vez que a função invoca a si mesma

gera uma nova entrada na pilha, gerando uma nova execução pendente, até que ocorra a condição de parada do algoritmo.



Você Sabia?

Quando há um encadeamento muito grande de chamadas de funções, pode ocorrer uma exceção do tipo **stack overflow**.

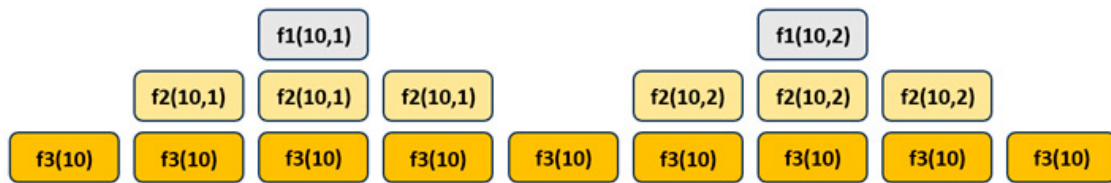
O uso da **call stack** é fundamental para a execução **síncrona**, pois garante o bloqueio do processo, enquanto o subprocesso é executado. O uso da pilha irá garantir o retorno correto, no momento certo, pois, caso contrário, poderíamos executar uma instrução sem que a informação necessária estivesse disponível.

Considere o código da listagem seguinte.

```
const f1 = (a,b) => a + b;
const f2 = (a,b) => {
  valor = f1(a,b);
  console.log(valor);
};
const f3 = (a) => {
  for(let i=1; i<=2; i++)
    f2(a,i);
}
f3(10);
```

A evolução da call Stack, durante a execução, pode ser observada a seguir.

Figura 1: Utilização da call stack para o código de exemplo



Fonte: Elaboração própria.

Como o JavaScript trabalha por padrão de forma **síncrona**, sem o uso de threads, ocorrerá a execução sequencial do código, com o **bloqueio** a cada nova função invocada. Isso significa que, se uma função demorar muito para retornar, ela bloqueará todo o fluxo de execução, podendo levar a erros, como **Timeout**, ou até à paralisação do servidor.

2. Funções de Callback e Chamadas Assíncronas

Uma alternativa ao processamento sequencial são as **funções assíncronas**, que executam em um **thread** separado, sem que ocorra o bloqueio do fluxo de execução original. O retorno de uma função assíncrona é do tipo **promise**, que representa uma esperança de valor, mas não a certeza, já que pode ocorrer um erro durante a execução.

Você pode observar um trecho de código assíncrono na listagem seguinte.

```
const somar = async (a,b) => a + b;
const imprimir_soma = async(a,b) => {
  let valor = await somar(a,b);
  console.log(valor);
  return "Processo Concluido";
}
imprimir_soma(1,2).then((retorno) => console.log(retorno));
```

A definição de uma função assíncrona é feita utilizando a palavra reservada **async**, garantindo a execução sem bloqueio e o retorno em um **promise**. Dentro de uma função assíncrona, pode ser utilizado o operador **await**, que irá esperar o retorno de outra função assíncrona, tirando a camada promise dos dados.

Embora você não possa utilizar o operador `await` no processamento síncrono, pode trabalhar com o operador **then**, que captura o retorno da função, remove a camada `promise` e permite especificar uma função de tratamento. Note que o uso de `then` não bloqueia, como ocorre com `await`, permitindo apenas efetuar o tratamento da informação quando ela for recebida.

As funções de tratamento são chamadas de **callback** e são a base para o comportamento assíncrono no NodeJS. Na prática, temos apenas uma função que fica preparada para receber os dados de um processo assíncrono, ao seu término, permitindo efetuar o tratamento necessário.

Você verá esse tipo de programação sendo utilizado para as mais diversas operações de I/O, como no tratamento de arquivos e chamadas HTTP, já que são demoradas e poderiam bloquear a execução do servidor. Na prática, será comum invocarmos a operação e fornecer a `callback` que irá lidar com os dados na conclusão do processo.

Um exemplo de leitura de arquivo assíncrona é apresentado a seguir.

```
const $fs = require('fs');
$fs.readFile('/Users/denis/teste.txt', 'utf8', (err, data) => {
  if(err) {
    console.error(err);
    return;
  }
  console.log(data);
});
```

Na leitura assíncrona do arquivo, você terá um retorno que pode ser um erro ou o texto do arquivo, sendo esses recebidos, respectivamente, nos parâmetros `err` e `data`. Caso ocorra o erro, ele será exibido no console, encerrando a execução da `callback` com `return`, e, no caso contrário, o texto será exibido.

Tome cuidado ao utilizar uma chamada como essa, pois ela não bloqueará a execução da linha seguinte no fluxo principal.

EXEMPLO

Se tivéssemos uma instrução na linha seguinte, após a chamada assíncrona, emitindo a mensagem "Leitura concluída", essa mensagem seria impressa enquanto o arquivo ainda estivesse sendo lido pelo método `readFile`.

Em geral, as funções assíncronas disponibilizadas nas bibliotecas do NodeJS retornam um primeiro parâmetro com o erro que pode ter ocorrido. Isso é muito importante para o programador, pois, durante o processamento assíncrono, ele não poderá utilizar a estrutura de controle de exceções padrão.

Você também pode utilizar o módulo de **fs/promises**, que permitirá adotar o modelo de tratamento de erros usual, desde que haja bloqueio com `await`.

```
const $fs = require('fs/promises');  
async function teste_leitura() {  
  try {  
    const data = await $fs.readFile('/Users/denis/teste.txt',  
      {encoding: 'utf8'});  
    console.log(data);  
  } catch (err) {  
    console.log(err);  
  }  
}  
teste_leitura();
```

3. Callback Queue e Event Loop

No ambiente do NodeJS, devido ao modelo de **thread único**, as funções de **callback** não são executadas imediatamente ao final do processo assíncrono. Na verdade, o processo concluído acrescenta sua função de callback em uma estrutura **FIFO** denominada **callback queue**, ou fila de callback, contendo todas as funções de tratamento prontas para execução.

Nesse ponto, temos a **call stack** para execução das funções, e sempre que a pilha de chamadas está vazia, a próxima função da **callback queue**, segundo a ordem de entrada, inicia a sua execução. Enquanto os processos assíncronos acrescentam suas funções de tratamento na fila, ao término da execução, o **event loop** fica responsável por recuperá-las.

Compreendendo o funcionamento da **callback queue**, você criará um código mais eficiente, tirando proveito do paralelismo, o que permitirá tratar múltiplas solicitações

simultaneamente. Esse é um requisito natural para os servidores HTTP, pois terão de lidar com diversos usuários, oferecendo o melhor tempo de resposta possível.



Destaque

Um cuidado que você deverá tomar é o de lidar com os erros nas funções callback, evitando efeitos indesejáveis na execução do servidor.

Um componente fundamental na arquitetura do NodeJS é o **event loop**, já que ele tem a responsabilidade de iniciar a execução das callbacks. Em termos práticos, ele é um elemento **single thread** que executa continuamente, olhando para a **callback queue** e a **call stack**, de forma a invocar a próxima função de callback sempre que a pilha de chamadas fica vazia, o que indica o término da execução para a função anterior.

Além da pilha de chamadas (call stack) e da fila de retorno (callback queue), o event loop utiliza outros componentes internos, como observadores de I/O e temporizadores. Como é um processo de thread único, ou seja, que executa uma função de cada vez, você precisa tomar cuidado ao definir suas funções de callback, evitando operações de I/O bloqueantes e processamentos longos, já que poderiam impedir a execução de outras funções de callback, causando problemas de desempenho e, até mesmo, paralisando todo o sistema.



Destaque

Procure utilizar sempre operações de I/O assíncronas e dividir operações longas em operações menores com comportamento assíncrono.

De acordo com o sistema de eventos adotado pelo NodeJS, também ocorre um consumo sequencial, em uma fila denominada **event queue**. Cada vez que ocorre o disparo de um evento, ele é colocado na fila e, ao consumir o evento, o **event loop** utiliza um thread do **thread pool** para executar as operações do processo assíncrono envolvido. Ao final da execução do processo, a função de callback, pronta para execução, é adicionada na **callback queue**.

Uma forma de utilizar o sistema de eventos é através do **EventEmitter**.

```
const EventEmitter = require('events');
const eventEmitter = new EventEmitter();
eventEmitter.on('valor', (valor) => {
  console.log(`Emitido o valor ${valor}`);
});
for(let i=1; i<=100; i++)
  eventEmitter.emit('valor',i);
```

Em termos práticos, definimos um evento e associamos uma função de callback para o valor fornecido. Ao emitir o evento, com a passagem do valor para a função de callback, ele será incluído na fila de eventos, para que possa ser consumido pelo event loop.

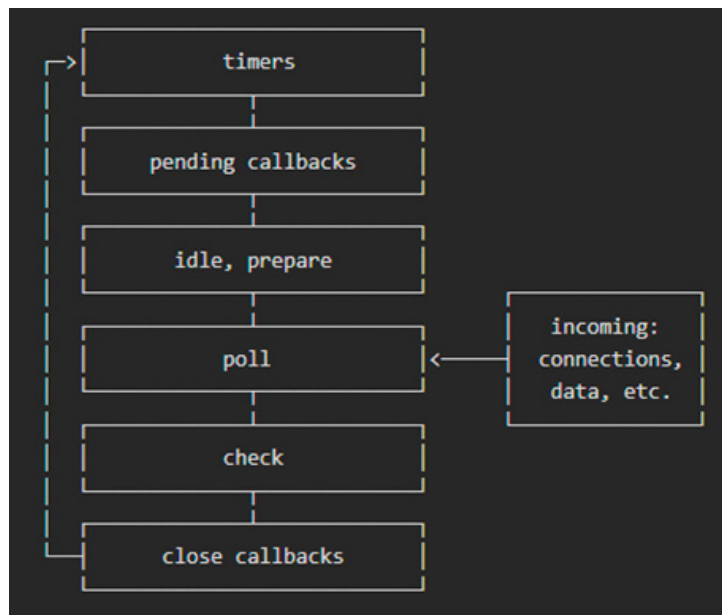
Muitas bibliotecas utilizam o EventEmitter para permitir aos programadores a implementação de respostas para alguns estados do processo assíncrono.

EXEMPLO

A biblioteca **PdfKit** permite definir o tratamento para o evento “**end**”, relacionado ao término da geração do conteúdo no formato PDF.

A execução do event loop segue um ciclo consistente de operações, no qual cada fase trata de um aspecto específico do comportamento assíncrono.

Figura 2: Ciclo de vida do event loop



Fonte: <https://nodejs.org/en/docs/guides/event-loop-timers-and-nexttick>

Cada bloco representa uma **fase** do ciclo de execução do **event loop**, com uma **fila de callbacks** para executar. Apesar de cada fase ter características próprias, quando a fase ocorre, operações específicas dela são executadas e, em seguida, as funções de callback são consumidas, até que a fila se esgote ou o limite de callbacks seja atingido, mudando para a próxima fase na sequência.

Note que qualquer operação pode agendar mais operações e novos eventos podem ser gerados na fase de **poll**, enquanto os eventos são processados. Como resultado, uma callback de execução longa pode fazer com que a fase de pool seja executada por mais tempo que o limite de um temporizador.

Na fase de **timer** são processados os temporizadores, especificados através dos métodos **setTimeout** e **setInterval**. No exemplo seguinte, devido à espera de 2 segundos, ocorrerá a impressão na ordem Início, Fim e Timeout.

```

console.log('Início');
setTimeout(() => {
  console.log('Timeout');
}, 2000);
console.log('Fim');
  
```


As funções de callback adiadas na iteração anterior do loop, como alguns tipos de erro, são processadas na fase **pending callbacks**, enquanto as tarefas de menor prioridade, como acionamento do coletor de lixo quando o servidor está ocioso, são executadas em **idle, prepare**, uma fase de uso interno.

Na fase de **poll** ocorre a verificação de novas operações de I/O e execução das funções da callback relacionadas, com exceção daquelas relacionadas ao evento **close**, gerenciadas por temporizadores ou iniciadas com **setImmediate**.

O tratamento do evento **data** normalmente ocorre nessa fase.

```
const fs = require('fs');
const readStream = fs.createReadStream('./file.txt');
readStream.on('data', (chunk) => {
  console.log(chunk.toString());
});
```

Prosseguindo para a fase de **check**, ocorre o processamento de qualquer callback adicionada com **setImmediate**, e a última fase, que é conhecida como **close callbacks**, executa callbacks para eventos do tipo **close**.

Você pode observar, na listagem seguinte, um exemplo de callback para o evento **close** no fechamento de um socket.

```
const net = require('net');
const server = net.createServer((socket) => {
  socket.on('close', () => {
    console.log('Socket fechado');
  });
});
server.listen(8000);
```

4. Usando Promise no NodeJS

Você poderá utilizar as diversas técnicas assíncronas descritas para obter a maior eficiência possível na execução de seus aplicativos NodeJS. Em diversas situações, o modelo de utilização é determinado pela biblioteca que foi adotada.

O retorno assíncrono pode ser definido pelo uso de classes **promise**, as quais podem retornar um valor de determinado tipo ou um erro ocorrido. Você poderá observar um primeiro exemplo na listagem seguinte.

```
const dividir = (a,b) => {  
  const promise = new Promise((resolve, reject)=>{  
    if(b==0){  
      reject("Divisao por zero");  
    } else {  
      resolve(a/b);  
    }  
  });  
  return promise;  
}  
  
for(let i = 2; i>=-2; i--)  
  dividir(10,i).then((x)=>console.log(x))  
  .catch((error)=>console.log(error));
```

Todo retorno definido em **resolve** será capturado na cláusula **then** ou por meio do operador **await**, enquanto os erros são gerados por **reject**, podendo ser capturados na cláusula **catch**.

Ao trabalhar com promise, você pode utilizar funções de forma encadeada, em que o resultado de uma pode alimentar a execução da função subsequente.

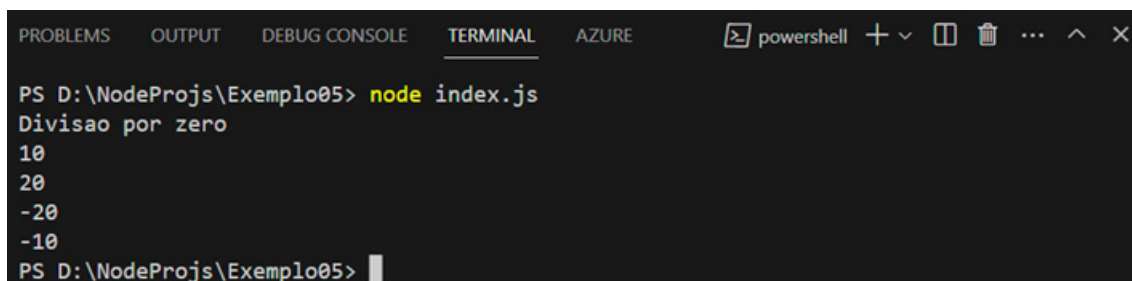
```
// O código de dividir não precisa ser modificado  
const dobrar = (x) => Promise.resolve(x*2);  
for(let i = 2; i>=-2; i--)
```

```
dividir(10, i).then((x1)=>dobrar(x1))
  .then((x2)=>console.log(x2))
  .catch((error)=>console.log(error));
```

No exemplo, após efetuar a divisão, que tem o resultado em x1, o método dobrar é invocado com o valor recebido, levando para a execução seguinte, com o resultado em x2, que é impresso, mostrando o dobro da divisão. Como temos uma cláusula catch, no caso de divisão por zero será emitida a mensagem.

A execução do código pode ser observada na figura seguinte.

Figura 3: Execução de chamadas then encadeadas



```
PS D:\NodeProjs\Exemplo05> node index.js
Divisao por zero
10
20
-20
-10
PS D:\NodeProjs\Exemplo05>
```

Fonte: Elaboração própria.

Você deve ter observado que a mensagem de erro foi impressa antes dos resultados das funções de callback para cada operador then, em vez de ficar entre os valores da sequência. Isso está relacionado ao ciclo de execução do event loop, definindo o tipo de callback que será executado a cada fase, o que acaba mudando a prioridade do tratamento de erros.

Considerações Finais da Aula

O modelo síncrono, no NodeJS, não difere de outras plataformas, pois utiliza uma pilha de chamadas, ou call stack, que armazena o ponto de retorno para cada processo que é iniciado a partir de outro. Quando o processo termina, ele é retirado da pilha, retornando o resultado e desbloqueando o chamador.

Já no modelo assíncrono, no qual temos os processos executando em threads separados, existem estruturas organizadas para garantir a sincronização no retorno e tratamento dos dados, através de funções de callback. Chamada de event queue, essa estrutura de fila garante que os tratamentos sejam iniciados na ordem correta.

Não podemos esquecer a importância do event loop, que gerencia toda a execução no ambiente, consumindo filas de eventos e de callbacks, ao longo das diferentes fases do seu ciclo de vida.

Definido o modelo de execução, a programação assíncrona em JavaScript será baseada no uso de promise, um tipo de dado que representa a esperança de uma resposta. Ocorrendo essa resposta, ela será tratada na função de callback, podendo, também, ser utilizada a assinatura async na função e o operador await para esperar o retorno no thread do processo assíncrono.

Apenas compreendendo esses elementos estaremos aptos a criar sistemas com acesso a dispositivos no NodeJS, já que é priorizado o modelo assíncrono.

Materiais Complementares

Artigo

Node.js Call Stack, Callback Queue, and Event Loop

2023, Vegibit.

O artigo sintetiza informações acerca de call stack, callback queue e event loop, além de trazer alguns exemplos de código JavaScript assíncrono.

Link para acesso: <https://vegibit.com/node-js-call-stack-callback-queue-and-event-loop/> (acesso em 12 set. 2023.)

Artigo

The Node.js Event Loop, Timers, and process.nextTick()

2023, NodeJS.

O artigo faz parte da documentação oficial do NodeJS e explica o funcionamento do event loop e seu ciclo de vida, além do uso de timers e outros elementos com programação assíncrona.

Link para acesso: <https://nodejs.org/en/docs/guides/event-loop-timers-and-nexttick> (acesso em 12 set. 2023.)

Referências

FLANAGAN, D. **JavaScript: o guia definitivo**. Porto Alegre: Bookman, 2014. Disponível em: <https://abre.ai/gKFR>. Acesso em: 12 set. 2023.

OLIVEIRA, C.; ZANETTI, H. **JavaScript descomplicado: programação para a Web, IoT e dispositivos móveis**. São Paulo: Érica, 2020. Disponível em: <https://abre.ai/gKFV>. Acesso em: 12 set. 2023.

OLIVEIRA, C.; ZANETTI, H. **Node.js: programe de forma rápida e prática**. São Paulo: Expressa, 2021. Disponível em: <https://abre.ai/gKFY>. Acesso em: 12 set. 2023.